Divide-and-Conquer

Divide-and-Conquer Technique

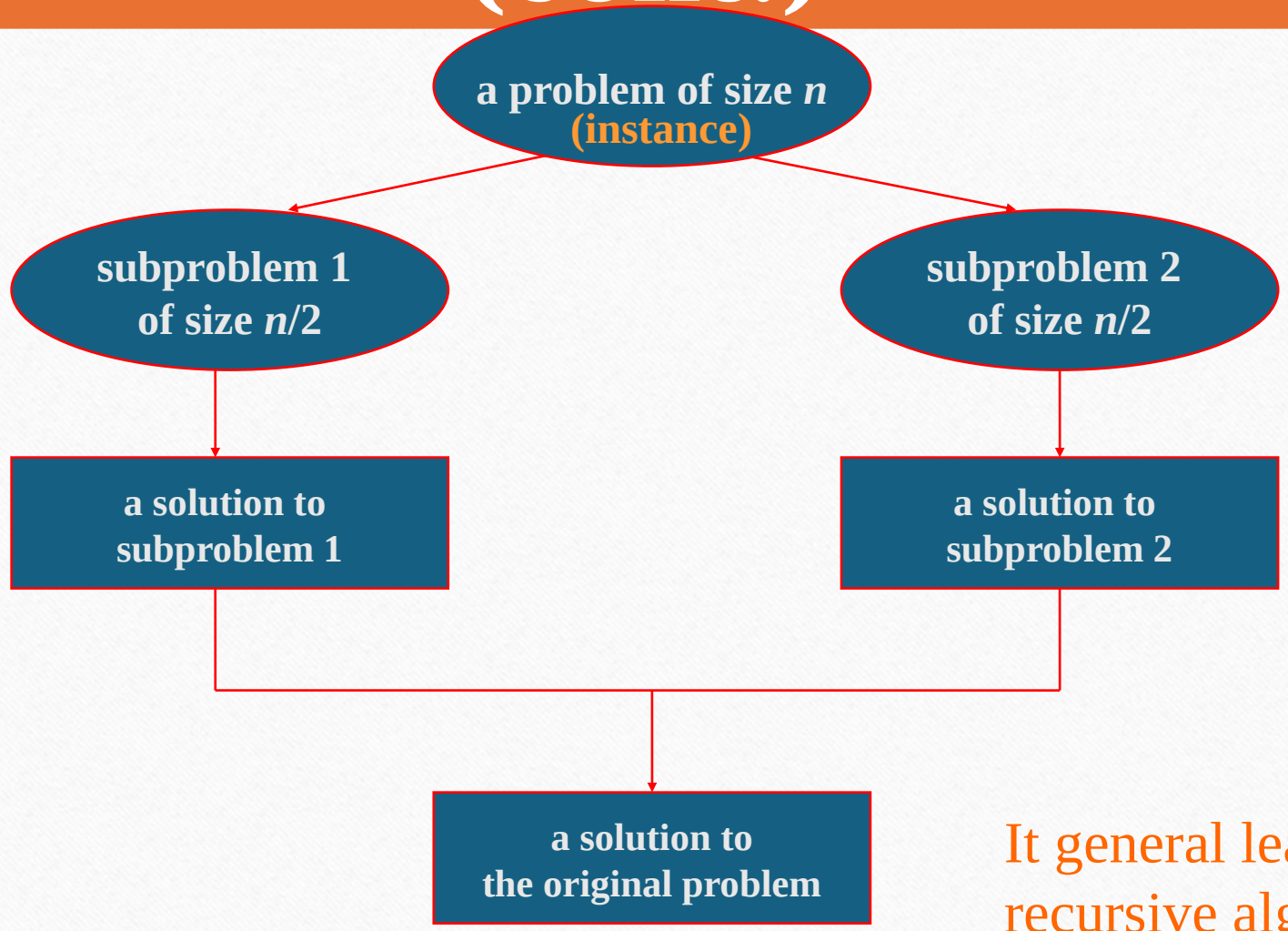
The most-well known algorithm design strategy:

1. Divide instance of problem into two or more smaller instances

2. Solve smaller instances recursively

3. Obtain solution to original (larger) instance by combining these solutions

Divide-and-Conquer Technique (cont.)



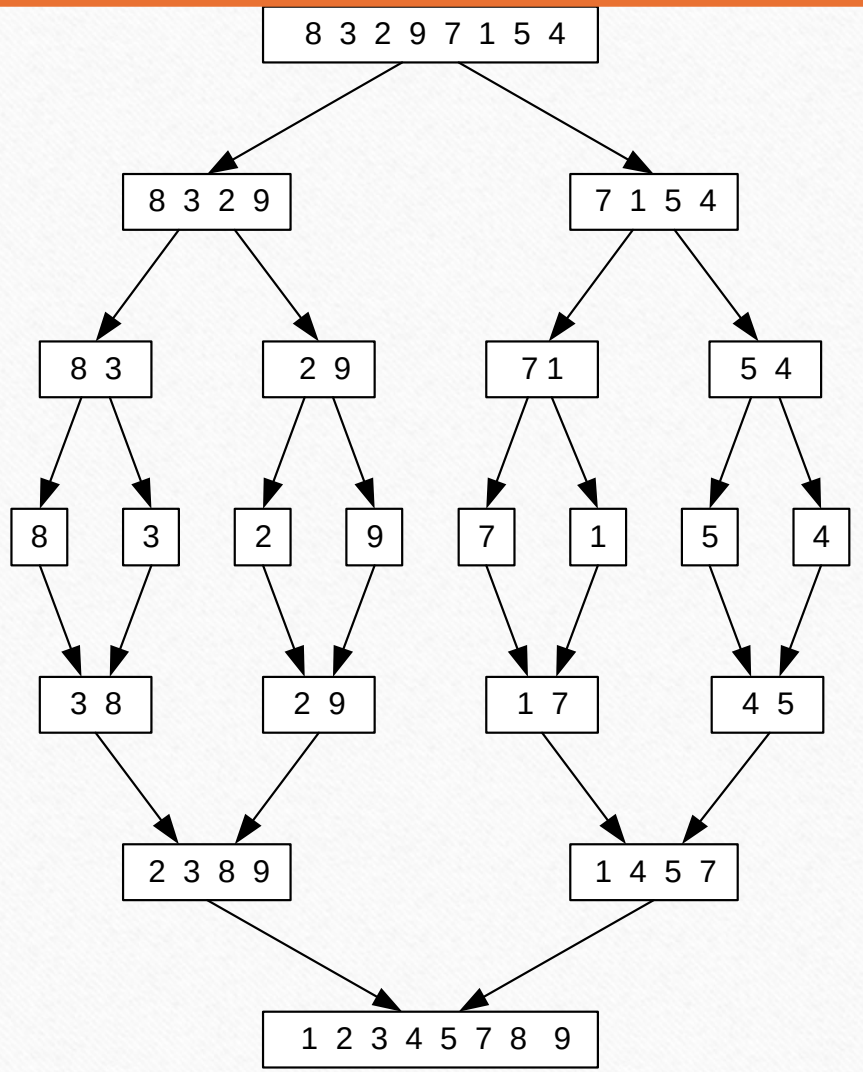
It general leads to a recursive algorithm!

Divide-and-Conquer Examples

- Merge sort
- Quicksort
- Matrix multiplication: Strassen’s algorithm
- Multiplication of large integers

Mergesort

Merge sort Example



Pseudocode of Mergesort

ALGORITHM *Mergesort*$(A[0..n-1])$

//Sorts array $A[0..n-1]$ by recursive mergesort
//Input: An array $A[0..n-1]$ of orderable elements
//Output: Array $A[0..n-1]$ sorted in nondecreasing order
if $n > 1$
    copy $A[0..\lfloor n/2 \rfloor - 1]$ to $B[0..\lfloor n/2 \rfloor - 1]$
    copy $A[\lfloor n/2 \rfloor..n-1]$ to $C[0..\lceil n/2 \rceil - 1]$
    Mergesort$(B[0..\lfloor n/2 \rfloor - 1])$
    Mergesort$(C[0..\lceil n/2 \rceil - 1])$
    Merge(B, C, A)

Pseudocode of Mergesort

ALGORITHM *Merge*$(B[0..p-1], C[0..q-1], A[0..p+q-1])$

```
//Merges two sorted arrays into one sorted array
//Input: Arrays B[0..p − 1] and C[0..q − 1] both sorted
//Output: Sorted array A[0..p + q − 1] of the elements of B and C
i ← 0; j ← 0; k ← 0
while i < p and j < q do
    if B[i] ≤ C[j]
        A[k] ← B[i]; i ← i + 1
    else A[k] ← C[j]; j ← j + 1
    k ← k + 1
if i = p
    copy C[j..q − 1] to A[k..p + q − 1]
else copy B[i..p − 1] to A[k..p + q − 1]
```

Time complexity: $\Theta(p + q) = \Theta(n)$ comparisons

Merge

- B=[2, 4, 15, 20] C= [1, 16, 100, 120] merge B &C in A
- i=0 j=0 k=0
- B[i]=2 > C[j] = 1 therefore A[0]=1 k=1, j=1
- B[i]=2 < c[j]=16 therefore A[1]= 2 k=2, i=1
- B[i]=4 < c[j]=16 A[2]=4 i=2, k = 3
- B[i]=15 < c[j]=16 A[3]= 15 i=3, k=4
- B[i]=20 > c[j]=16 A[4]= 16 k=4, j= 2
- B[i]=20 < c[2]=100 A[5] = 20 k=5 , i=4
- i>3 therefore copy remaining element of C in A
- A[6]=100 A[7]=120
- A[] =[1, 2, 4, 15, 16, 20, 100, 120]

Analysis of Mergesort

Let T(n) denote time complexity of sorting n elements using merge sort then :

$T(n) = 2T(n/2) + cn, T(1) = 0$ for some constant c

Analysis of Merge sort

For simplification purpose assume $n = 2^k$ for some positive constant k

$T(n) = 2T(n/2) + Cn$, where c is a constant

$T(n) = 2[2T(n/2^2) + Cn/2] + Cn = 2^2\ T(n/\ 2^2) + 2Cn$

$T(n) = 2^2[2T(n/2^3) + Cn/2^2] + 2Cn$

$T(n) = 2^3\ T(n/\ 2^3) + 3Cn$

……..

……..

……..

$T(n) = 2^k\ T(n/\ 2^k) + kCn$

Therefore, $T(n) = 2^k\ T(1) + kCn = k\ Cn$ (since T(1)=0)

Which implies T (n) = O (kn) = O(n logn)

[since $n = 2^k$, k = log n]

Analysis of Merge sort (Using Masters Theorem)

$T(n) = aT(n/b) + f(n)$ where $f(n)$ $\Theta(n^d)$, $d \geq 0$

Master Theorem: If $a < b^d$, $T(n)$ $\Theta(n^d)$

If $a = b^d$, $T(n)$ $\Theta(n^d \log n)$

If $a > b^d$, $T(n)$

$T(n) = 2T(n/2) + cn$, $T(1) = 0$ for some constant c

Comparing the Recurrence Relation with the Masters Theorem we get
$a = 2$, $b=2$, $d=1$
Using Masters Theorem
$a = b^d$

Therefore $T(n)$ $\Theta(n^d \log n)$

=> $T(n)$ $\Theta(n \log n)$

Quick Sort

Quick Sort

- Small instance has n <= 1. Every small instance is a sorted instance.
- To sort a large instance, select a pivot element from out of the n elements.
- Partition the n elements into 3 groups left, middle and right.
- The middle group contains only the pivot element.
- All elements in the left group are <= pivot.
- All elements in the right group are >= pivot.
- Sort left and right groups recursively.
- Answer is sorted left group, followed by middle group followed by sorted right group.

Example

Use 6 as the pivot.

Sort left and right groups recursively.

Choice Of Pivot

- Pivot is leftmost element in list that is to be sorted.
  - When sorting a[6:20], use a[6] as the pivot.
- Randomly select one of the elements to be sorted as the pivot.
  - When sorting a[6:20], generate a random number r in the range [6, 20]. Use a[r] as the pivot.

Choice Of Pivot

- Median-of-Three rule. From the leftmost, middle, and rightmost elements of the list to be sorted, select the one with median key as the pivot.
  - When sorting a[6:20], examine a[6], a[13] ((6+20)/2), and a[20]. Select the element with median (i.e., middle) key.

Choice Of Pivot

Median

Complexity

- $O(n)$ time to partition an array of n elements.
- Let $t(n)$ be the time needed to sort n elements.
- $t(0) = t(1) = d$, where d is a constant.
- When $t > 1$,

  $t(n) = t(|left|) + t(|right|) + cn$,

  where c is a constant.
- $t(n)$ is maximum when either $|left| = 0$ or $|right| = 0$ following each partitioning.

Complexity

- This happens, for example, when the pivot is always the smallest or largest element.
- For the worst-case time,

  $t(n) = t(n-1) + cn, n > 1$
- Use repeated substitution to get $t(n) = O(n^2)$.
- The best case arises when |left| and |right| are equal (or differ by 1) following each partitioning.
- For the best case, the recurrence is the same as for merge sort. $T(n) = 2T(n/2) + Cn$, where c is a constant

Analysis of Quicksort

Best case: split in the middle (same as merge sort)— O(n log n)

Worst case: sorted array! — $O(n^2)$

$T(n) = T(n-1) + \Theta(n)$, $T(1) = 0$

$T(n) = T(n-1) + cn = T(n-2) + c(n-1) + cn$

.....

.....

$T(n) = T(1) + c(n + (n-1) + \ldots + 2) = O(n^2)$

Average case: random arrays — $\Theta(n \log n)$?

Analysis of Quick sort Best Case (Using Masters Theorem)

$T(n) = aT(n/b) + f(n)$ where $f(n)$ $\Theta(n^d)$, $d \geq 0$

Master Theorem: If $a < b^d$, $T(n)$ $\Theta(n^d)$

If $a = b^d$, $T(n)$ $\Theta(n^d \log n)$

If $a > b^d$, $T(n)$

$T(n) = 2T(n/2) + Cn$

Comparing the Recurrence Relation with the Masters Theorem we get
$a = 2, b=2, d=1$
Using Masters Theorem
$a = b^d$

Therefore $T(n)$ $\Theta(n^d \log n)$

$\Rightarrow$ $T(n)$ $\Theta(n \log n)$

Average Time Complexity of Quick Sort

- Let t(n) denote average time complexity of sorting n elements using quick sort
- For n <=1, t(n) = d , for some constant d.
- t(n) <= cn + 1/n [$\sum_0^{n-1}$ (t(s) + t(n-s-1)], where s and n-s-1 denote number of elements in the left segment and the right segment respectively.

$$t(n) \leq cn + \frac{2}{n}\sum_{s=0}^{n-1} t(s) \leq cn + \frac{4d}{n} + \frac{2}{n}\sum_{s=2}^{n-1} t(s)$$

Average Time Complexity of Quick Sort

Proof by induction: We show that t(n) <= kn logn, where k= 2(c+d)

Base case n=2, t(2) <= 2(c+d)

Assume that theorem is true for n<m , now to prove the theorer

$$t(m) \leq cm + \frac{4d}{m} + \frac{2}{m}\sum_{s=2}^{m-1} t(s) \leq cm + \frac{4d}{m} + \frac{2k}{m}\sum_{s=2}^{m-1} s \log_e s$$

Average Time Complexity of Quick Sort

- $s \log_e s$ is an increasing function of s.
- $\int_2^m s \log_e s ds < \frac{m^2 \log_e m}{2} - \frac{m^2}{4}$.

Using these facts and Equation 18.12, we obtain

$$\begin{aligned} t(m) &< cm + \frac{4d}{m} + \frac{2k}{m} \int_2^m s \log_e s ds \\ &< cm + \frac{4d}{m} + \frac{2k}{m} \left[\frac{m^2 \log_e m}{2} - \frac{m^2}{4} \right] \\ &= cm + \frac{4d}{m} + km \log_e m - \frac{km}{2} \\ &< km \log_e m \end{aligned}$$

Matrix Multiplication

Conventional Matrix Multiplication

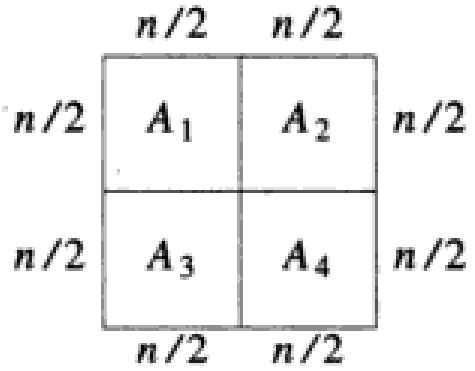
Example [matrix multiplication] The product of two $n \times n$ matrices A and B is a third $n \times n$ matrix C where $C(i,j)$ is given by

$$C(i,j) = \sum_{k=1}^{n} A(i,k) * B(k,j), \quad 1 \le i \le n, \quad 1 \le j \le n \qquad (18.1)$$

```
for (int i=0; i< n ; i++)
for (int j=0; j<n; j++)
      {
        c[i][j]=0;
        for (int k=0; i<n; k++)
          c[i][j]+= A[i][k]*B[k][j];
        }
```

$O(n^3)$

Matrix Multiplication : Divide and Conquer



(a) Dividing A into four

$$\begin{bmatrix} A_1 & A_2 \\ A_3 & A_4 \end{bmatrix} * \begin{bmatrix} B_1 & B_2 \\ B_3 & B_4 \end{bmatrix} = \begin{bmatrix} C_1 & C_2 \\ C_3 & C_4 \end{bmatrix}$$

(b) $A * B = C$

$$\begin{aligned} C_1 &= A_1B_1 + A_2B_3 \\ C_2 &= A_1B_2 + A_2B_4 \\ C_3 &= A_3B_1 + A_4B_3 \\ C_4 &= A_3B_2 + A_4B_4 \end{aligned}$$

T(n) = 8 T(n/2) + cn^2 for n>=2 and T(1)=d

Solve ! ?

$T(n) = aT(n/b) + f(n)$

where $f(n)$ $\Theta(n^d)$, d 0

Master's Theorem:

If $a < b^d$, $T(n)$ $\Theta(n^d)$

If $a = b^d$, $T(n)$ $\Theta(n^d \log n)$

If $a > b^d$, $T(n)$ $\Theta(n^{\log}$

- $T(n) = 8\ T(n/2) + cn^2 = 8\ [8T(n/2^2) + cn^2/4] + cn^2$
- $= 8^2T(n/2^2) + cn^2\ [1 + 2]$
- $= 8^2[8T(n/2^3) + cn^2/16] + cn^2\ [1 + 2 + 4]$
- $= 8^3T(n/2^3) + cn^2\ [1 + 2 + 2^2]$

- ……
- ……
- $= 8^kT(n/2^k) + cn^2\ [1 + 2+ 2^2 +\ldots + 2^{k-1}] =$ $O(n^3)$ (Since $n= 2^k$)

Analysis of Matrix Multiplication (Using Masters Theorem)

$T(n) = aT(n/b) + f(n)$ where $f(n)$ $\Theta(n^d)$, $d \geq 0$

Master Theorem: If $a < b^d$, $T(n)$ $\Theta(n^d)$

If $a = b^d$, $T(n)$ $\Theta(n^d \log n)$

If $a > b^d$ $T(n)$

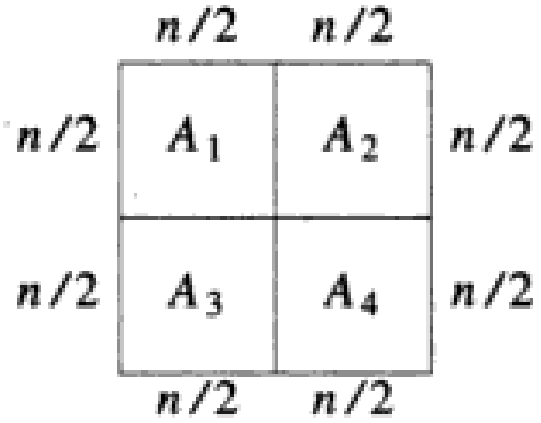
$$T(n) = 8\,T(n/2) + cn^2$$

Comparing the Recurrence Relation with the Masters Theorem we get
$a = 8, b=2, d=2$

Using Masters Theorem
$a > b^d$

Therefore $T(n)$ $\Theta(n^{\log_b a})$
=> $T(n)$ $\Theta(n^{\log_2 8}) = \Theta(n^3)$

Strassen's Matrix Multiplication



(a) Dividing A into four

$$\begin{bmatrix} A_1 & A_2 \\ A_3 & A_4 \end{bmatrix} * \begin{bmatrix} B_1 & B_2 \\ B_3 & B_4 \end{bmatrix} = \begin{bmatrix} C_1 & C_2 \\ C_3 & C_4 \end{bmatrix}$$

(b) $A * B = C$

Formulas for Strassen's Algorithm

$$
\begin{aligned}
D &= A_1(B_2 - B_4) \\
E &= A_4(B_3 - B_1) \\
F &= (A_3 + A_4)B_1 \\
G &= (A_1 + A_2)B_4 \\
H &= (A_3 - A_1)(B_1 + B_2) \\
I &= (A_2 - A_4)(B_3 + B_4) \\
J &= (A_1 + A_4)(B_1 + B_4)
\end{aligned}
$$

$$
\begin{aligned}
C_1 &= E + I + J - G \\
C_2 &= D + G \\
C_3 &= E + F \\
C_4 &= D + H + J - F
\end{aligned}
$$

$T(n) = 7\,T(n/2) + cn^2$ $\quad$ $T(2) = d$, for some constant d

Analysis of Strassen's Algorithm

$T(n) = 7\,T(n/2) + cn^2$ $T(1) = d$, for some constant d

$T(n) = 7\,T(n/2) + cn^2 = 7\,[7\,T(n/2^2) + c(n/2)^2\,] + cn^2 =$

$= 7^2\,T(n/2^2) + cn^2\,[1 + 7/4]$

$= 7^3\,T(n/2^3) + cn^2\,[1 + 7/4 + (7/4)^2]$

.....

.....

$= 7^k\,T(n/2^k) + cn^2\,[1 + 7/4 + (7/4)^2 + \ldots (7/4)^{k-1}]$

$= 7^k\,T(n/2^k) + cn^2\,(7/4)^k / [7/4 - 1] = O(7^k)$

$= O(7^{\log n}) = O(n^{\log 7}) = O(n^{2.81})$

Analysis of Strassen's Algorithm (Using Masters Theorem)

$T(n) = aT(n/b) + f(n)$ where $f(n)$ $\Theta(n^d)$, $d \geq 0$

Master Theorem: If $a < b^d$, $T(n)$ $\Theta(n^d)$

If $a = b^d$, $T(n)$ $\Theta(n^d \log n)$

If $a > b^d$ $T(n)$

$T(n) = 7\ T(n/2) + cn^2$

Comparing the Recurrence Relation with the Masters Theorem we get
$a = 7, b=2, d=2$

Using Masters Theorem
$a > b^d$

Therefore $T(n)$ $\Theta(n^{\log_b a})$
$\Rightarrow$ $T(n)$ $\Theta(n^{\log_2 7})$

General Divide-and-Conquer Recurrence

$T(n) = aT(n/b) + f(n)$ where $f(n) \in \Theta(n^d)$, $d \geq 0$

Master Theorem: If $a < b^d$, $T(n) \in \Theta(n^d)$

If $a = b^d$, $T(n) \in \Theta(n^d \log n)$

If $a > b^d$, $T(n) \in \Theta(n^{\log_b a})$

Note: The same results hold with O instead of Θ.

Examples: $T(n) = 4T(n/2) + n$ $T(n)$?

$T(n) = 4T(n/2) + n^2$ $T(n)$?

$T(n) = 4T(n/2) + n^3$ $T(n)$?

Example: Strassen's Matrix Multiplication

$$
\begin{aligned}
D &= A_1(B_2 - B_4) \\
E &= A_4(B_3 - B_1) \\
F &= (A_3 + A_4)B_1 \\
G &= (A_1 + A_2)B_4 \\
H &= (A_3 - A_1)(B_1 + B_2) \\
I &= (A_2 - A_4)(B_3 + B_4) \\
J &= (A_1 + A_4)(B_1 + B_4)
\end{aligned}
$$

$$
\begin{aligned}
C_1 &= E + I + J - G \\
C_2 &= D + G \\
C_3 &= E + F \\
C_4 &= D + H + J - F
\end{aligned}
$$

$$
\begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix} * \begin{bmatrix} 5 & 6 \\ 7 & 8 \end{bmatrix} =
$$

$$
\begin{aligned}
D &= 1(6 - 8) = -2 \\
E &= 4(7 - 5) = 8
\end{aligned}
$$

$$
\begin{aligned}
F &= (3 + 4)5 = 35 \\
G &= (1 + 2)8 = 24 \\
H &= (3 - 1)(5 + 6) = 22 \\
I &= (2 - 4)(7 + 8) = -30 \\
J &= (1 + 4)(5 + 8) = 65
\end{aligned}
$$

$$
\begin{aligned}
C_1 &= 8 - 30 + 65 - 24 = 19 \\
C_2 &= -2 + 24 = 22 \\
C_3 &= 8 + 35 = 43 \\
C_4 &= -2 + 22 + 65 - 35 = 50
\end{aligned}
$$

Exercise

Multiply the following two matrix using Strassen's Matrix Multiplication Method. Multiply 2 x 2 matrix conventionally.

5	10	15	20
5	10	15	20
1	2	3	4
5	6	7	8

1	2	3	4
5	6	7	8
9	10	11	12
13	14	15	16

450	500	550	600
450	500	550	600
90	100	110	120
202	228	254	280

Exercice

Solve the following recurrence equations using substitution Method and Master's theorem. In all the examples considered take t(1)=1:

$$t(n) = 10t(n/3) + 11n,\ n \geq 3 \text{ and a power of } 3.$$

$$t(n) = 128t(n/2) + 6n^8,\ n \geq 2 \text{ and a power of } 2.$$

Large Integer Multiplication using Divide and Conquer Approach

Multiplying two n-digit numbers (radix $r = 2, 10$)

$0 \leq x, y < r^n$

$$
\begin{aligned}
x &= x_1 \cdot r^{n/2} + x_0 \qquad x_1 = \text{high half} \\
y &= y_1 \cdot r^{n/2} + y_0 \qquad x_0 = \text{low half} \\
0 &\leq x_0, x_1 < r^{n/2} \\
0 &\leq y_0, y_1 < r^{n/2}
\end{aligned}
$$

$$z = x \cdot y = x_1 y_1 \cdot r^n + (x_0 \cdot y_1 + x_1 \cdot y_0) r^{n/2} + x_0 \cdot y_0$$

Karatsuba's Method

Let

$$
\begin{aligned}
z_0 &= \underline{x_0 \cdot y_0} \\
z_2 &= x_1 \cdot y_1 \\
z_1 &= (x_0 + x_1) \cdot (y_0 + y_1) - z_0 - z_2 \\
&= x_0 y_1 + x_1 y_0 \\
z &= z_2 \cdot r^n + z_1 \cdot r^{n/2} + z_0
\end{aligned}
$$

Time Complexity

$$
\begin{aligned}
T(n) &= \text{time to multiply two } n\text{-digit } \sharp' s \\
&= 3T(n/2) + \theta(n) \\
&= \theta\left(n^{\log_2 3}\right) = \theta\left(n^{1.5849625\cdots}\right)
\end{aligned}
$$

For $A = a_1a_0 = 2345$,

hence, $a_1 = 23$ and $a_0 = 45$ and

$B = b_1b_0 = 0678$,

hence, $b_1 = 06$ and $b_0 = 78$

$c_2 = a_1 * b_1$

$= 23 * 06$

$= 138$

$c_0 = a_0 * b_0$

$= (45 * 78)$

$= 3510$

$c_1 = (a_1 + a_0) * (b_1 + b_0) - (c_2 + c_0)$

$= (23 + 45) * (06 + 78) - (138 + 3510)$

$= 68 * 84 - 3648$

$= 2064$

$$C = c_2 * 10^4 + c_1 * 10^2 + c_0$$

$$= 1380000 + 206400 + 3510 = 15, 89, 910$$

Tutorial

Q1. Multiply the following two matrix using Strassen's Matrix Multiplication Method. Multiply 2 x 2 matrix conventionally.

5	3	1	2
5	1	5	2
1	2	3	4
1	1	5	5

1	1	3	3
5	5	2	2
2	5	2	5
1	1	1	1

$$
\begin{aligned}
D &= A_1(B_2 - B_4) \\
E &= A_4(B_3 - B_1) \\
F &= (A_3 + A_4)B_1 \\
G &= (A_1 + A_2)B_4 \\
H &= (A_3 - A_1)(B_1 + B_2) \\
I &= (A_2 - A_4)(B_3 + B_4) \\
J &= (A_1 + A_4)(B_1 + B_4)
\end{aligned}
$$

$$
\begin{aligned}
C_1 &= E + I + J - G \\
C_2 &= D + G \\
C_3 &= E + F \\
C_4 &= D + H + J - F
\end{aligned}
$$

Q2. Solve the following using Master's theorem

$$t(n) = 128t(n/2) + 6n^8, n \geq 2 \text{ and a power of 2.}$$